Homework 4: Mathematics Behind Computer Graphics

Team Members:
Aitzhanov Tamerlan(IT-2408)
Kartabaev Tair(IT-2408)
Sagadenov Miras(IT-2408)

Course: Analytic Geometry
Team Name: **ASQABAK**
Date: November 7, 2025

Contents

1 Introduction

Mathematics is the foundation of realistic 3D computer graphics. From simple rotations to advanced lighting and shadow algorithms, every rendered image depends on precise mathematical computations. This report presents the essential mathematical and algorithmic ideas behind 3D rendering: how a camera operates, how transformations shape objects, how light interacts with materials, and how shadows form.

2 Camera in 3D Space

A virtual camera defines the viewer's perspective in a 3D scene. The camera model involves transforming world coordinates into view and projection coordinates. Lengyel (2012) defines several coordinate systems—object, world, camera, and clip space—that form the **rendering pipeline** [1].

2.1 Coordinate Transformations

Vertices of a 3D model are first stored in *object space*. To display them correctly, they must be transformed successively through world, camera, and projection spaces:

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = P \cdot V \cdot M \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

where M, V, and P are the model, view, and projection matrices respectively.

2.2 Perspective Projection

According to Lengyel (2012, Ch. 5.5), a perspective projection maps the view frustum into homogeneous clip space using a 4×4 projection matrix:

$$P = \begin{bmatrix} \frac{1}{\tan(\theta/2)} & 0 & 0 & 0 \\ 0 & \frac{1}{\tan(\theta/2)} & 0 & 0 \\ 0 & 0 & \frac{f+n}{n-f} & \frac{2fn}{n-f} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

which simulates how distant objects appear smaller [2].

2.3 Algorithm (Pseudocode)

Listing 1: Camera Transformation Algorithm

```
Input: Object vertices, camera position, orientation
Output: 2D projected coordinates

1. Build Model matrix M
2. Build View matrix V using camera position/direction
3. Build Projection matrix P (perspective)
```

```
4. For each vertex:
       a. Transform = P * V * M * vertex
       b. Perform perspective divide
       c. Map result to screen coordinates
```

3 Construction and Transformation of 3D Figures

3D objects are represented by vertices and faces in local coordinate systems. According to Lengyel's *Chapter 4: Transforms*, transformations between spaces (object → world → camera) are performed using linear and affine matrices [1].

3.1 Transformation Matrices

Translation:

$$T(x, y, z) = \begin{bmatrix} 1 & 0 & 0 & x \\ 0 & 1 & 0 & y \\ 0 & 0 & 1 & z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Rotation about z-axis:

$$R_z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Scaling:

$$S(s_x, s_y, s_z) = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

3.2 Algorithm

Listing 2: 3D Object Transformation Algorithm

```
For each 3D object:
    1. Define vertices in local coordinates
    2. Apply S (scaling), R (rotation), and T (translation)
    3. Multiply: world_vertex = T * R * S * vertex
```

4 Light Reflection and Material Properties

Lengyel (2012, Ch. 7) explains that lighting involves determining how light interacts with surfaces using reflection models such as Lambert's and Phong's models [3].

4.1 Diffuse Reflection

$$I_d = k_d \cdot I_L \cdot \max(0, \mathbf{N} \cdot \mathbf{L})$$

4.2 Specular Reflection

$$I_s = k_s \cdot I_L \cdot (\max(0, \mathbf{R} \cdot \mathbf{V}))^n$$

4.3 Ambient Reflection

$$I_a = k_a \cdot I_L$$

4.4 Combined Model

$$I = I_a + I_d + I_s$$

4.5 Algorithm

Listing 3: Lighting Computation Algorithm

```
For each visible surface:
    Compute N, L, V vectors
    Id = kd * IL * max(0, dot(N, L))
    R = 2 * dot(N, L) * N - L
    Is = ks * IL * (max(0, dot(R, V)))^n
    I = Ia + Id + Is
```

5 Shadows and Light Sources

Lengyel's Chapter 10 discusses **shadow mapping**, a key method for realistic shadow creation [4].

5.1 Shadow Mapping Concept

Shadow mapping renders the scene from the light's point of view, storing depths in a *shadow map*. During rendering, each pixel's depth is compared to this map to determine if it's in shadow.

Listing 4: Shadow Mapping Algorithm

```
1. Render scene from light's␣perspective␣to␣create␣depth␣map
2.␣For␣each␣pixel:
␣␣␣␣␣␣␣␣If␣pixel_depth␣>␣light_depth_map␣   ␣pixel␣is␣shadowed
```

6 Conclusion

Mathematics provides the foundation for every step of the rendering pipeline—from transforming coordinates to simulating light behavior. Understanding these principles allows for accurate and efficient implementation of 3D graphics without relying blindly on libraries.

7 References

References

[1] Lengyel, E. (2012). *Mathematics for 3D Game Programming and Computer Graphics* (4th ed.), Chapter 4 – Transforms. CRC Press.

[2] Lengyel, E. (2012). *Mathematics for 3D Game Programming and Computer Graphics*, Chapter 5 – Perspective Projections. CRC Press.

[3] Lengyel, E. (2012). *Mathematics for 3D Game Programming and Computer Graphics*, Chapter 7 – Lighting and Shading. CRC Press.

[4] Lengyel, E. (2012). *Mathematics for 3D Game Programming and Computer Graphics*, Chapter 10 – Shadow Mapping. CRC Press.

[5] Foley, J. D., van Dam, A., Feiner, S. K., Hughes, J. F. (2013). *Computer Graphics: Principles and Practice.* Addison-Wesley.

[6] Shirley, P. (2018). *Fundamentals of Computer Graphics.* CRC Press.